

Hardware-Praktikum 2026

O. Amft und L. Häusler, Intelligent Embedded Systems Lab,
Institute for Computer Science, Albert-Ludwigs-University Freiburg, Freiburg im Breisgau

Exercise sheet 1 (29 points)

Welcome

Welcome to the Hardware Lab Course! Throughout the course we will gradually introduce the hardware of the *Seeed Studio XIAO nRF52840 Sense*. Use the time before the submission deadline to familiarise yourself with Arduino. Check the borrowed hardware for any defects as early as possible and contact your tutor if you encounter problems.

We strongly encourage you to use the templates for this exercise sheet as it may offer more guidance and will simplify the structure of your code.

Objective:

Using the microcontroller, you will interface with various hardware components. You will get to know the Arduino IDE, control the built-in RGB LED, read a gas sensor via I2C, and visualise measurement data on an OLED display.

Prologue:

Good news: This year comes with brand new hardware!

As a first task, you can unbox your components and sensors and get familiar with your kit. Please carefully unpack the microcontroller, expansion board, and gas sensor from their packaging. Then, unbox the acrylic case and mount it onto the expansion board to help protect your setup from accidental damage. The result should look something like Fig. 1

You may keep the original packaging for extra protection during the course. However, there is no requirement to return all packaging at the end of the course.

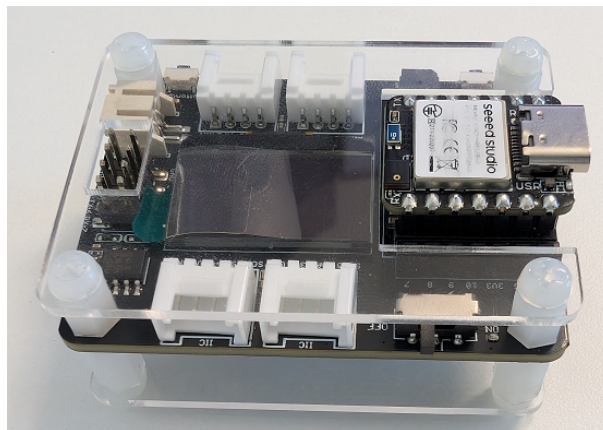


Figure 1: Microcontroller and the expansion board with the acrylic glass cover mounted

Download and install the Arduino IDE (Version 2.x) from the Arduino website. Open *File* → *Preferences* and add the following URL to the *Additional Board Manager URLs* field: https://files.seeedstudio.com/arduino/package_seeeduino_boards_index.json. Then open *Tools* → *Board* → *Board Manager*, search for *Seeed nRF52* and install the package. Select **Seeed XIAO nRF52840 Sense** under *Tools* → *Board*. Also install the following libraries via the Library Manager (*Tools* → *Manage Libraries*):

- **Adafruit TinyUSB**
- **U8g2** (by olikraus)

Windows: To install the driver for the USB programmer, open Device Manager and locate the *USB Serial Port* (probably under *Other Devices*). Right-click on the *USB Serial Port* and select *Update Driver Software*. The required driver can be found in the Arduino folder (subfolder *drivers*) or under:

`C:\Users\USERNAME\AppData\Local\Arduino15\packages\arduino\hardware\avr\1.8.x\drivers`.

After installation, the *USB Serial Port* should be listed under *Ports (COM & LPT)*. If not, try rebooting.

Linux (Ubuntu): The USB programmer should be automatically detected (*Ltd FT232 USB-Serial (UART) IC*). However, access is only available to users in the *dialout* group. Therefore, add your user to this group and restart to be able to program the Arduino.

macOS: No driver installation is necessary. Select the port with the name/path: `/dev/cu.usbserial-A<XXXX>`.

The Arduino is programmed with a subset of the C++ language. Each program consists of a *setup()* and a *loop()* function. The code in the setup function is executed once after power-up, reset or programming. After that, the loop function is called in an infinite loop.

Note: Use a data cable, not a charge-only cable. Some USB-C cables do not support data transfer and will cause upload failures.

Task 1 (6 points): LEDs and Timing

The XIAO nRF52840 Sense has a built-in RGB LED. Each colour is on a separate pin, accessible via the constants `LED_RED`, `LED_GREEN`, and `LED_BLUE`. **Important:** the LED is *active-low* — write `LOW` to turn it on and `HIGH` to turn it off.

- i) Make the red LED blink at 1 Hz (1 second on, 1 second off) using the `delay()` function. (2 points)
- ii) Implement the same blinking using the `millis()` function, without using `delay()`. (2 points)
- iii) What is the advantage of method ii) over method i)? Write a short answer as a comment in your code. (2 points)
- iv) **Bonus task:** Extend the code from ii): after 10 complete blink cycles the LED should automatically change colour (red → green → blue → red ...). (1 point)

Note: Since the XIAO nRF52840 uses a native USB chip, the serial connection must be explicitly awaited. Add the following at the start of `setup()`.

```
#include <Adafruit_TinyUSB.h>
// ...
Serial.begin(115200);
while (!Serial); // Wait until the Connection is established.
```

Task 2 (8 points): I2C Sensor Readout

Microcontrollers rarely work alone — they must communicate with sensors, displays, and other peripherals. Three protocols appear constantly in embedded systems:

UART (Universal Asynchronous Receiver/Transmitter) uses two wires (TX and RX) for point-to-point communication. No clock line is shared; both sides agree on a *baud rate* in advance. This is what `Serial.print()` uses internally.

1-Wire (Dallas/Maxim) uses a *single* data line. Devices can even draw their operating power from the data line (“parasite power”). Each device carries a unique 64-bit ROM address burned in at the factory, so many sensors can share one wire. It is simple to wire but slow and mostly limited to simple sensors (e.g. DS18B20 temperature sensors).

I2C (Inter-Integrated Circuit, also called “Two-Wire Interface”) uses exactly two shared lines:

- **SDA** — Serial Data: carries all data bits in both directions.
- **SCL** — Serial Clock: always driven by the *controller* to synchronise every transfer.

Both lines are *open-drain*: devices can only pull the line low; external *pull-up resistors* to V_{CC} return it to high when no device is driving it. Many devices can share the same two wires simultaneously; each is identified by a unique **7-bit address** (giving up to 112 usable device addresses on one bus). The device that initiates communication is called the **controller** (here: the microcontroller); all others are **peripherals** (sensor, display, ...).

Note: In the sensor’s datasheet, these are referred to as “Master” and “Slave”.

A write transaction (controller → peripheral):

1. controller issues a **START** condition: SDA falls while SCL is high.
2. controller transmits the 7-bit device **address** + a **Write** bit (0).
3. Addressed peripheral replies with an **ACK**: pulls SDA low for one clock. Any other device stays silent. If no device answers, the controller receives a **NACK** — useful for detecting whether a device is present.
4. controller sends one or more **data bytes**; peripheral ACKs each.
5. controller issues a **STOP**: SDA rises while SCL is high. Bus is free again.

A read transaction (peripheral → controller): identical start sequence but with a **Read** bit (1). The peripheral then drives SDA and clocks in data bytes. The controller sends a final **NACK** before STOP to tell the peripheral it has received all bytes.

Arduino Wire library — mapping to I2C:

Wire call	Corresponding I2C action
Wire.begin()	Initialise I2C hardware (call once in setup())
Wire.beginTransmission(addr)	START + 7-bit address + Write bit
Wire.write(byte)	Queue one data byte
Wire.endTransmission()	Transmit queued bytes + STOP; returns 0 on ACK
Wire.requestFrom(addr, n)	START + address + Read bit; clock in <i>n</i> bytes + STOP
Wire.read()	Return the next buffered received byte

In this exercise, you will read a VOC/eCO₂ gas sensor (SGP30) via the I2C bus. The Arduino *Wire* library provides I2C communication.

You will also have to find and use information from the sensor's datasheet, a copy is downloadable from ILIAS (datasheet section). For those who are unfamiliar with navigating a datasheet, you can find a short explanation on page 5.

Firstly, plug the Grove VOC and eCO₂ sensor into the appropriate connection using the included cable (Hint: read the connection labels on the expansion board).

- i) Write an **I2C scanner**: scan all addresses from 8 to 127 (the first 7 addresses are taken by default) using the *Wire* library and print all found device addresses in hexadecimal format on the Serial monitor. Note down the addresses you find. (2 points)

- ii) **Datasheet cross-check.** Cross-reference the address(es) you found with the SGP30 datasheet.

- Which address belongs to the SGP30?
- What is the two-byte command code for initialization?
- What is the two-byte command code for measurement?
- How many bytes does a measurement response contain? What does each byte represent?

Write your answers as comments in your code. (2 points)

- iii) **Helper functions.** Implement the two functions below using *Wire*. These abstractions will keep your later code clean and readable. (2 points)

```
// Sends a 2-byte command (MSB first) to the SGP30.
void sgp30_cmd(uint8_t msb, uint8_t lsb);

// Reads exactly n bytes from the SGP30.
// Returns true on success, false if fewer bytes were received.
bool sgp30_read(uint8_t n);
```

Resource note: Why uint8_t and not int? On this platform int occupies 4 bytes — four times more memory than a single byte requires. Always choose the smallest type that fits your data.

- iv) **Initialise and read.** Using your helper functions: send the initialization command in setup(), wait for the warm-up period, then in loop() send the measurement command once per second, read the response bytes, parse the eCO₂ (ppm) and TVOC (ppb) values, and print both on the Serial Monitor. Use millis() for timing — avoid delay() in the main loop, unless necessary e.g. sensor response delay. (2 points)

Note: The SGP30 requires a warm-up period of approximately 15 seconds, during which it returns default values (eCO₂ = 400 ppm, TVOC = 0 ppb). This is normal behaviour. Also keep in mind the specified timings for the initialization and measurement, provided in the datasheet.

Task 3 (6 points): OLED Display

The XIAO expansion board includes a 128×64-pixel OLED display that communicates over the *same* I2C bus used in Task 2. Use the **U8g2** library with the following constructor:

```
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE);
```

Every screen update follows this three-step pattern:

```
u8g2.clearBuffer();    // 1. Erase the internal pixel buffer
// ... drawing commands ...
u8g2.sendBuffer();     // 2. Push the buffer to the physical display
```

- i) Initialise the display and show the text "Hardware Praktikum" on startup. (3 points)
- ii) Extend your program from Task 2: display the eCO₂ and TVOC values on the OLED, updating every second alongside your Serial output. (3 points)

Note: The y coordinate in `setCursor()` describes the text *baseline*, not the top edge. With an 8 px font, start at e.g. y = 10 for the first line.

Task 4 (8 points): Interactive Visualisation

In this final exercise, you will graphically visualise the eCO₂ reading on the OLED display. Use Arduino's `map()` function to map the sensor value to pixel values:

```
long map(long value, long fromLow, long fromHigh, long toLow, long toHigh);
```

A sensible value range for the SGP30 is 400 ppm (fresh air) to 2000 ppm (poor indoor air quality), but test different values to see what fits better. You can also use a higher polling rate, i.e. shorten the time between measurements to get a more fluid response.

- i) Map the eCO₂ value to a percentage (0–100 %) and display it on the screen. Use `constrain()` to ensure the percentage never exceeds the interval [0, 100], even when the sensor returns values outside the expected range.:

```
constrain(value, min, max) //clamps a value within [min, max]
```

(4 points)

- ii) Additionally represent the percentage as a **filled bar** on the display. The bar should use the full width of the display. The function `u8g2.drawBox()` makes this straightforward.

(4 points)

- iii) **Bonus task:** Make the visualisation more creative. Ideas: a smiley face reflecting air quality; a circular progress bar; any other graphical representation of your choice. (Bonus)

Note: Breathe out on the sensor to raise the CO₂ value and test your visualisation in real time.

Submission

Archive your sketches in a ZIP file and upload it on ILIAS. Your submission must contain exactly **two** sketches:

- **Sketch 1:** Task 1
- **Sketch 2:** Tasks 2–4

Task parts that have been superseded by later sub-tasks should be **commented out**, not deleted. Check your archive for completeness before submitting.

How to Read a Datasheet

A datasheet is the authoritative technical reference for any component. Before writing a single line of code for a new sensor, find and open its datasheet. Here is a systematic approach:

- 1. Device address.** Search for “I2C address” or “controller/target(“slave”) address” — typically in an *Electrical Characteristics* or *Interface* section. It may be given in hexadecimal, binary, or both.
- 2. Command structure.** Sensor commands are listed in a table showing the command name, its *command code* (one or two bytes you write to the sensor), and the expected response length. Check whether the code is sent MSB-first or LSB-first. For the SGP30, commands are always two bytes, MSB first.
- 3. Response format.** After sending a command and waiting the required time, you read back a fixed number of bytes. The datasheet shows their order: which bytes carry the measurement (MSB, then LSB of the value) and which are *CRC-8 checksums* for error detection. Identify each byte’s role before writing your parsing code.
- 4. Timing.** Always check the maximum command execution time (t_{max}) before reading back a response. Reading too soon returns stale or invalid data.
- 5. Initialisation sequence.** Many sensors require one or more commands on power-up before they deliver valid measurements. Look for a “start-up” or “initialisation” procedure. Missing this step often causes the sensor to return default placeholder values.

For this exercise: use the SGP30 datasheet (Sensirion). Chapter 6 describes the I2C interface and command set.